LABORATORY RECORD

B.Tech - Artificial Intelligence & Machine Learning

II Year B.Tech. AI & ML - I Semester

JNTUA R23 Regulations

(23A05302P)

ADVANCED DATA STRUCTURES & ALGORITHM ANALYSIS LAB

L T P C = 0 0 3 1.5

Programs with Source Code and Output

Name : Bharath

Roll No : _____

Branch : AI & ML Section : _____

INDEX

S.No	Name of the Program	Page
1	AVL Tree - Insert and Delete using a file	3
2	B-Tree of order 5 - search, insert and delete	6
3	Min Heap and Max Heap using arrays	10
4	BFT and DFT of a graph (Adjacency Matrix and Adjacency Lists)	12
5	Bi-connected components of a graph	15
6	Quick Sort and Merge Sort - execution time	17
7	Single Source Shortest Paths (Greedy) - Matrix vs Lists	19
8	Job Sequencing with Deadlines (Greedy)	22
9	0/1 Knapsack Problem using Dynamic Programming	24
10	N-Queens Problem using Backtracking	26
11	0/1 Knapsack Problem using Backtracking	28
12	Travelling Salesperson Problem using Branch and Bound	30

Program 1

AVL Tree – Insert and Delete using a file

Aim:

To construct an AVL tree from a set of elements stored in a file, perform insert and delete operations on it and write the contents of the tree into a new file using in-order traversal.

Algorithm:

1. Read the elements from input.txt and insert them one by one.
2. Insert as in a BST, then update height and balance factor (BF = h(left) – h(right)) of every node on the path back.
3. If BF > 1 or BF < -1 rotate: LL -> right rotation, RR -> left rotation, LR / RL -> double rotation.
4. Delete: remove like a BST (replace by in-order successor if two children) and re-balance on the way up.
5. Do an in-order traversal and write the keys into output.txt.

Program:

```
#include <bits/stdc++.h>
using namespace std;
struct Node{ int key,h; Node *l,*r;
  Node(int k):key(k),h(1),l(0),r(0){} };
int ht(Node*n){ return n?n->h:0; }
int bal(Node*n){ return n?ht(n->l)-ht(n->r):0; }
void upd(Node*n){ n->h=1+max(ht(n->l),ht(n->r)); }
Node* rotR(Node*y){
  Node*x=y->l; y->l=x->r; x->r=y; upd(y); upd(x); return x; }
Node* rotL(Node*x){
  Node*y=x->r; x->r=y->l; y->l=x; upd(x); upd(y); return y; }
Node* fix(Node*n){
  upd(n); int b=bal(n);
  if(b>1){ if(bal(n->l)<0) n->l=rotL(n->l); return rotR(n); }
  if(b<-1){ if(bal(n->r)>0) n->r=rotR(n->r); return rotL(n); }
  return n; }
Node* ins(Node*n,int k){
  if(!n) return new Node(k);
  if(k<n->key) n->l=ins(n->l,k);
```

```
  else if(k>n->key) n->r=ins(n->r,k);
  return fix(n); }
Node* del(Node*n,int k){
  if(!n) return n;
  if(k<n->key) n->l=del(n->l,k);
  else if(k>n->key) n->r=del(n->r,k);
  else{
    if(!n->l||!n->r){ Node*t=n->l?n->l:n->r; delete n; return t; }
    Node*s=n->r; while(s->l) s=s->l;
    n->key=s->key; n->r=del(n->r,s->key); }
  return fix(n); }
void inorder(Node*n,ofstream&f){
  if(!n) return;
  inorder(n->l,f); f<<n->key<<" "; inorder(n->r,f); }
void show(Node*n,int sp){      // tree printed sideways (root at left)
  if(!n) return;
  show(n->r,sp+6); cout<<string(sp,' ')<<n->key<<"\n"; show(n->l,sp+6); }
int main(){
  ofstream("input.txt")<<"30 20 40 10 25 35 50 5 15 22 27";
  ifstream in("input.txt"); Node*root=0; int x;
  while(in>>x) root=ins(root,x);
  cout<<"AVL tree built from input.txt (height "<<ht(root)<<"):\n";
  show(root,0);
  root=ins(root,12);  root=del(root,40);
  cout<<"\nAfter insert 12 and delete 40 (height "<<ht(root)<<"):\n";
  show(root,0);
  ofstream out("output.txt"); inorder(root,out); out.close();
  ifstream r("output.txt"); string s; getline(r,s);
  cout<<"\nIn-order written to output.txt:\n"<<s<<"\n";
}
```

Output:

```
AVL tree built from input.txt (height 4):
            50
      40
            35
30
                  27
            25
                  22
      20
                  15
            10
                  5

After insert 12 and delete 40 (height 4):
            50
                  35
      30
                  27
            25
                  22
20
            15
                  12
      10
            5

In-order written to output.txt:
5 10 12 15 20 22 25 27 30 35 50
```

Result: The program was executed successfully and the output is verified.

Program 2

B-Tree of order 5 – search, insert and delete

Aim:

To construct a B-Tree of order 5 with 100 random elements stored in an array and to implement searching, insertion and deletion operations.

Algorithm:

1. Order 5 means: max 5 children, max 4 keys and min 2 keys per node (except root).
2. Search: find the first key >= x in the node; if equal -> found, else go to that child.
3. Insert: go down to the leaf and insert the key in sorted order. If a node has 5 keys, split it and push the middle key up to the parent.
4. Delete: if the key is in an internal node replace it by its predecessor. If a child falls below 2 keys, borrow from a sibling or merge with it.
5. If the root becomes empty, its only child becomes the new root.

Program:

```
#include <bits/stdc++.h>
using namespace std;
const int MAXK=4, MINK=2;          // B-Tree of order 5
struct BNode{ vector<int> k; vector<BNode*> c; bool leaf=true; };
BNode* root=new BNode;
int pos(BNode*n,int x){
 return lower_bound(n->k.begin(),n->k.end(),x)-n->k.begin(); }
bool lookup(BNode*n,int x){
 int i=pos(n,x);
 if(i<(int)n->k.size()&&n->k[i]==x) return true;
 return n->leaf?false:lookup(n->c[i],x); }
bool add(BNode*n,int x,int&mid,BNode*&nr){
 int i=pos(n,x);
 if(n->leaf) n->k.insert(n->k.begin()+i,x);
 else{
  int m; BNode*r;
  if(add(n->c[i],x,m,r)){
   n->k.insert(n->k.begin()+i,m);
   n->c.insert(n->c.begin()+i+1,r); } }
```

```
  if((int)n->k.size()<=MAXK) return false;       // overflow: split
  mid=n->k[2]; nr=new BNode; nr->leaf=n->leaf;
  nr->k.assign(n->k.begin()+3,n->k.end()); n->k.resize(2);
  if(!n->leaf){
    nr->c.assign(n->c.begin()+3,n->c.end()); n->c.resize(3); }
  return true; }
void insert(int x){
  int m; BNode*r;
  if(add(root,x,m,r)){
    BNode*t=new BNode; t->leaf=false; t->k={m}; t->c={root,r}; root=t; } }
void fixup(BNode*n,int i){          // child i has too few keys
  BNode*c=n->c[i];
  if(i>0&&n->c[i-1]->k.size()>MINK){          // borrow from left
    BNode*l=n->c[i-1];
    c->k.insert(c->k.begin(),n->k[i-1]);
    n->k[i-1]=l->k.back(); l->k.pop_back();
    if(!l->leaf){ c->c.insert(c->c.begin(),l->c.back()); l->c.pop_back(); } }
  else if(i+1<(int)n->c.size()&&n->c[i+1]->k.size()>MINK){  // from right
    BNode*r=n->c[i+1];
    c->k.push_back(n->k[i]); n->k[i]=r->k[0]; r->k.erase(r->k.begin());
    if(!r->leaf){ c->c.push_back(r->c[0]); r->c.erase(r->c.begin()); } }
  else{                                  // merge with a sibling
    int j=i>0?i-1:i; BNode*a=n->c[j],*b=n->c[j+1];
    a->k.push_back(n->k[j]);
    a->k.insert(a->k.end(),b->k.begin(),b->k.end());
    a->c.insert(a->c.end(),b->c.begin(),b->c.end());
    n->k.erase(n->k.begin()+j); n->c.erase(n->c.begin()+j+1); delete b; } }
void rem(BNode*n,int x){
  int i=pos(n,x); bool f=i<(int)n->k.size()&&n->k[i]==x;
  if(f&&n->leaf){ n->k.erase(n->k.begin()+i); return; }
  if(n->leaf) return;
  if(f){                              // replace by predecessor
    BNode*p=n->c[i]; while(!p->leaf) p=p->c.back();
    n->k[i]=p->k.back(); x=n->k[i]; }
  rem(n->c[i],x);
  if((int)n->c[i]->k.size()<MINK) fixup(n,i); }
```

```
void erase(int x){
  rem(root,x);
  if(root->k.empty()&&!root->leaf) root=root->c[0]; }
void inorder(BNode*n,vector<int>&v){
  for(size_t i=0;i<n->k.size();i++){
    if(!n->leaf) inorder(n->c[i],v);
    v.push_back(n->k[i]); }
  if(!n->leaf) inorder(n->c.back(),v); }
void print(vector<int>&v){
  for(size_t i=0;i<v.size();i++) cout<<setw(4)<<v[i]<<((i%10==9)?"\n":"");
  cout<<"\n"; }
int main(){
  vector<int> a(500); iota(a.begin(),a.end(),1);
  mt19937 g(2026); shuffle(a.begin(),a.end(),g); a.resize(100);
  cout<<"100 random elements stored in array:\n"; print(a);
  for(int x:a) insert(x);
  int h=1; for(BNode*n=root;!n->leaf;n=n->c[0]) h++;
  cout<<"B-Tree (order 5) built. Height = "<<h<<"\n";
  int key[]={a[7],a[55],999};
  for(int x:key)
    cout<<"Search "<<x<<" : "<<(lookup(root,x)?"Found":"Not found")<<"\n";
  insert(777);
  cout<<"Inserted 777, search 777 : "<<(lookup(root,777)?"Found":"Not found")<<"\n";
  int d[]={a[3],a[20],a[40],a[60],a[80]};
  for(int x:d){ erase(x); cout<<"Deleted "<<x<<"\n"; }
  cout<<"Search "<<d[0]<<" after delete : "
      <<(lookup(root,d[0])?"Found":"Not found")<<"\n";
  vector<int> v; inorder(root,v);
  cout<<"\nIn-order of final tree ("<<v.size()<<" keys):\n"; print(v);
}
```

Output:

```
100 random elements stored in array:
 209  200  394  406  395  492   91  472   17  164
 217  420  287  418  235  403  197  474  282  211
 191  459  483  315  253  461   29  163  331   39
```

```
  68 326 305 489 320 286 364 226 132  82
  46  84 148 173 324 456 280  55  34 248
 187 309  76 484 190 219  35 341   3  93
 100 178 335 318   9  64 131 346  50  60
 268 204 469 149 107  22 203 370 386 347
 397  15 375 291 199 411 233  51  54 321
 360  71 239  92 277 337 127 443 482 342

B-Tree (order 5) built. Height = 4
Search 472 : Found
Search 219 : Found
Search 999 : Not found
Inserted 777, search 777 : Found
Deleted 406
Deleted 191
Deleted 46
Deleted 100
Deleted 397
Search 406 after delete : Not found

In-order of final tree (96 keys):
   3   9  15  17  22  29  34  35  39  50
  51  54  55  60  64  68  71  76  82  84
  91  92  93 107 127 131 132 148 149 163
 164 173 178 187 190 197 199 200 203 204
 209 211 217 219 226 233 235 239 248 253
 268 277 280 282 286 287 291 305 309 315
 318 320 321 324 326 331 335 337 341 342
 346 347 360 364 370 375 386 394 395 403
 411 418 420 443 456 459 461 469 472 474
 482 483 484 489 492 777
```

Result: The program was executed successfully and the output is verified.

Program 3

Min Heap and Max Heap using arrays

Aim:

To construct Min and Max heaps using arrays, delete any element from the heap and display the contents of the heap.

Algorithm:

1. Store the heap in an array: parent of i is (i-1)/2, children are 2i+1 and 2i+2.
2. Insert: add at the end and move it up (swap with parent) while it violates the heap order.
3. Delete any element: find its index, replace it by the last element, reduce the size, then move it up or down to restore the order.
4. Display the array after every operation.

Program:

```
#include <bits/stdc++.h>
using namespace std;
struct Heap{
  vector<int> a; bool mn;            // mn=true : Min Heap
  Heap(bool m):mn(m){}
  bool better(int x,int y){ return mn? x<y : x>y; }
  void up(int i){
    while(i>0&&better(a[i],a[(i-1)/2])){
      swap(a[i],a[(i-1)/2]); i=(i-1)/2; } }
  void down(int i){
    int n=a.size();
    while(1){
      int l=2*i+1,r=l+1,b=i;
      if(l<n&&better(a[l],a[b])) b=l;
      if(r<n&&better(a[r],a[b])) b=r;
      if(b==i) break;
      swap(a[i],a[b]); i=b; } }
  void insert(int x){ a.push_back(x); up(a.size()-1); }
  void remove(int x){
    int i=find(a.begin(),a.end(),x)-a.begin();
    if(i==(int)a.size()){ cout<<x<<" not found\n"; return; }
```

```
    a[i]=a.back(); a.pop_back();
    if(i<(int)a.size()){ up(i); down(i); } }
  void show(){ for(int x:a) cout<<x<<" "; cout<<"\n"; }
};
int main(){
  int d[]={40,10,30,50,20,60,5,45};
  Heap mn(true),mx(false);
  for(int x:d){ mn.insert(x); mx.insert(x); }
  cout<<"Min Heap : "; mn.show();
  cout<<"Max Heap : "; mx.show();
  mn.remove(30); mx.remove(50);
  cout<<"\nMin Heap after deleting 30 : "; mn.show();
  cout<<"Max Heap after deleting 50 : "; mx.show();
  mn.remove(5);  mx.remove(60);
  cout<<"\nMin Heap after deleting root 5   : "; mn.show();
  cout<<"Max Heap after deleting root 60  : "; mx.show();
  mn.remove(99);
}
```

Output:

```
Min Heap : 5 20 10 45 40 60 30 50
Max Heap : 60 45 50 40 20 30 5 10

Min Heap after deleting 30 : 5 20 10 45 40 60 50
Max Heap after deleting 50 : 60 45 30 40 20 10 5

Min Heap after deleting root 5   : 10 20 50 45 40 60
Max Heap after deleting root 60  : 45 40 30 5 20 10
99 not found
```

Result: The program was executed successfully and the output is verified.

Program 4

BFT and DFT of a graph (Adjacency Matrix and Adjacency Lists)

Aim:

To implement Breadth First Traversal and Depth First Traversal of a given graph when the graph is represented by (a) adjacency matrix (b) adjacency lists.

Algorithm:

1. BFT: mark the start vertex, push it into a queue; repeatedly remove a vertex, print it and enqueue all its unvisited neighbours.
2. DFT: visit the vertex, mark it and recursively visit each unvisited neighbour.
3. Matrix: neighbours of u are all v with mat[u][v] = 1. Lists: neighbours are the elements of adj[u].
4. Time: O(V^2) with matrix, O(V+E) with lists.

Program:

```
#include <bits/stdc++.h>
using namespace std;
const int N=7;
int mat[N][N]; vector<int> adj[N];
void bfsM(int s){
 bool vis[N]={0}; queue<int> q; q.push(s); vis[s]=1;
 while(!q.empty()){
  int u=q.front(); q.pop(); cout<<u<<" ";
  for(int v=0;v<N;v++)
   if(mat[u][v]&&!vis[v]){ vis[v]=1; q.push(v); } } }
void dfsM(int u,bool vis[]){
 vis[u]=1; cout<<u<<" ";
 for(int v=0;v<N;v++) if(mat[u][v]&&!vis[v]) dfsM(v,vis); }
void bfsL(int s){
 bool vis[N]={0}; queue<int> q; q.push(s); vis[s]=1;
 while(!q.empty()){
  int u=q.front(); q.pop(); cout<<u<<" ";
  for(int v:adj[u]) if(!vis[v]){ vis[v]=1; q.push(v); } } }
void dfsL(int u,bool vis[]){
 vis[u]=1; cout<<u<<" ";
 for(int v:adj[u]) if(!vis[v]) dfsL(v,vis); }
```

```
int main(){
 int e[][2]={{0,1},{0,2},{1,3},{1,4},{2,5},{4,6},{5,6}};
 for(auto&x:e){
  mat[x[0]][x[1]]=mat[x[1]][x[0]]=1;
  adj[x[0]].push_back(x[1]); adj[x[1]].push_back(x[0]); }
 cout<<"Adjacency Matrix:\n";
 for(int i=0;i<N;i++){
  for(int j=0;j<N;j++) cout<<mat[i][j]<<" ";
  cout<<"\n"; }
 cout<<"\nAdjacency Lists:\n";
 for(int i=0;i<N;i++){
  cout<<i<<" -> "; for(int v:adj[i]) cout<<v<<" "; cout<<"\n"; }
 bool v1[N]={0},v2[N]={0};
 cout<<"\n(a) Adjacency Matrix\nBFT : "; bfsM(0);
 cout<<"\nDFT : "; dfsM(0,v1);
 cout<<"\n\n(b) Adjacency Lists\nBFT : "; bfsL(0);
 cout<<"\nDFT : "; dfsL(0,v2); cout<<"\n";
}
```

Output:

```
Adjacency Matrix:
0 1 1 0 0 0 0
1 0 0 1 1 0 0
1 0 0 0 0 1 0
0 1 0 0 0 0 0
0 1 0 0 0 0 1
0 0 1 0 0 0 1
0 0 0 0 1 1 0

Adjacency Lists:
0 -> 1 2
1 -> 0 3 4
2 -> 0 5
3 -> 1
4 -> 1 6
5 -> 2 6
6 -> 4 5

(a) Adjacency Matrix
BFT : 0 1 2 3 4 5 6
DFT : 0 1 3 4 6 5 2

(b) Adjacency Lists
BFT : 0 1 2 3 4 5 6
DFT : 0 1 3 4 6 5 2
```

Result: The program was executed successfully and the output is verified.

Program 5

Bi-connected components of a graph

Aim:

To write a program for finding the bi-connected components (and articulation points) in a given graph.

Algorithm:

1. Do a DFS and store disc[u] (discovery time) and low[u] (lowest disc reachable using one back edge).
2. Push every edge on a stack while exploring.
3. After returning from child v of u, if low[v] >= disc[u] then u is an articulation point and the edges on the stack up to (u,v) form one bi-connected component.
4. Root is an articulation point only if it has more than one child.

Program:

```
#include <bits/stdc++.h>
using namespace std;
const int N=8;
vector<int> g[N]; int disc[N],low[N],timer_=0,cnt=0;
bool ap[N]; stack<pair<int,int>> st;
void dfs(int u,int p){
 disc[u]=low[u]=++timer_; int ch=0;
 for(int v:g[u]){
  if(v==p) continue;
  if(!disc[v]){
   st.push({u,v}); ch++; dfs(v,u);
   low[u]=min(low[u],low[v]);
   if(low[v]>=disc[u]){
    if(p!=-1||ch>1) ap[u]=1;
    cout<<"Component "<<++cnt<<" : ";
    pair<int,int> e;
    do{ e=st.top(); st.pop();
       cout<<"("<<e.first<<","<<e.second<<") "; }
    while(e!=make_pair(u,v));
    cout<<"\n"; } }
  else if(disc[v]<disc[u]){
   st.push({u,v}); low[u]=min(low[u],disc[v]); } } }
```

```
int main(){
  int e[][2]={{0,1},{0,2},{1,2},{1,3},{3,4},{4,5},{3,5},{5,6},{6,7}};
  for(auto&x:e){ g[x[0]].push_back(x[1]); g[x[1]].push_back(x[0]); }
  cout<<"Bi-connected components (edges):\n";
  dfs(0,-1);
  cout<<"Articulation points : ";
  for(int i=0;i<N;i++) if(ap[i]) cout<<i<<" ";
  cout<<"\nTotal components = "<<cnt<<"\n";
}
```

Output:

```
Bi-connected components (edges):
Component 1 : (6,7)
Component 2 : (5,6)
Component 3 : (5,3) (4,5) (3,4)
Component 4 : (1,3)
Component 5 : (2,0) (1,2) (0,1)
Articulation points : 1 3 5 6
Total components = 5
```

Result: The program was executed successfully and the output is verified.

Program 6

Quick Sort and Merge Sort - execution time

Aim:

To implement Quick sort and Merge sort and observe the execution time for various input sizes (best, average and worst cases).

Algorithm:

1. Quick sort: take first element as pivot, put smaller elements to its left and larger to its right, sort both parts recursively.
2. Merge sort: divide the array into halves, sort each half and merge the two sorted halves.
3. Best case for quick sort: pivot is always the median. Average: random data. Worst: already sorted data (O(n^2)).
4. Merge sort is O(n log n) in all cases. Time is measured with chrono for n = 1000, 2000, 4000, 8000.

Program:

```
#include <bits/stdc++.h>
using namespace std;
typedef vector<int> VI;
void qs(VI&a,int l,int r){
 if(l>=r) return;
 int p=a[l]; VI L,G;
 for(int i=l+1;i<=r;i++) (a[i]<p?L:G).push_back(a[i]);
 int k=l; for(int x:L) a[k++]=x;
 a[k]=p; int pi=k++; for(int x:G) a[k++]=x;
 qs(a,l,pi-1); qs(a,pi+1,r); }
void ms(VI&a,int l,int r){
 if(l>=r) return;
 int m=(l+r)/2; ms(a,l,m); ms(a,m+1,r);
 VI t; int i=l,j=m+1;
 while(i<=m&&j<=r) t.push_back(a[i]<=a[j]?a[i++]:a[j++]);
 while(i<=m) t.push_back(a[i++]);
 while(j<=r) t.push_back(a[j++]);
 copy(t.begin(),t.end(),a.begin()+l); }
void best(int lo,int hi,VI&o){      // builds best case for quick sort
 if(lo>hi) return;
```

```
 int m=(lo+hi)/2; o.push_back(m); best(lo,m-1,o); best(m+1,hi,o); }
double run(void(*f)(VI&,int,int),VI a){
 auto s=chrono::high_resolution_clock::now();
 f(a,0,a.size()-1);
 return chrono::duration<double,micro>(
      chrono::high_resolution_clock::now()-s).count(); }
int main(){
 mt19937 g(1);
 printf("Time in microseconds\n");
 printf("%6s | %-26s | %-26s\n","n","Quick Sort","Merge Sort");
 printf("%6s | %8s %8s %8s | %8s %8s %8s\n","",
      "Best","Avg","Worst","Best","Avg","Worst");
 for(int n=1000;n<=8000;n*=2){
  VI srt(n),rnd,rev,bst; iota(srt.begin(),srt.end(),0);
  rnd=srt; shuffle(rnd.begin(),rnd.end(),g);
  rev=srt; reverse(rev.begin(),rev.end());
  best(0,n-1,bst);
  printf("%6d | %8.0f %8.0f %8.0f | %8.0f %8.0f %8.0f\n",n,
   run(qs,bst),run(qs,rnd),run(qs,srt),
   run(ms,srt),run(ms,rnd),run(ms,rev)); }
}
```

Output:

```
Time in microseconds
     n | Quick Sort                          | Merge Sort
       |      Best       Avg      Worst |      Best       Avg      Worst
  1000 |        61       101       9528 |        65       133         59
  2000 |        97       242       9054 |       124       211        118
  4000 |       208       392      24021 |       240       524        276
  8000 |       437       817     106264 |       512       917        471
```

Result: The program was executed successfully and the output is verified.

Program 7

Single Source Shortest Paths (Greedy) - Matrix vs Lists

Aim:

To compare the performance of Single Source Shortest Paths using the greedy method (Dijkstra) when the graph is represented by an adjacency matrix and by adjacency lists.

Algorithm:

1. Set dist[source] = 0 and dist of all others = infinity.
2. Repeatedly pick the unfinished vertex u with the smallest dist (greedy choice) and mark it done.
3. Relax every edge (u,v): if dist[u] + w < dist[v] then dist[v] = dist[u] + w.
4. Matrix version picks the minimum by scanning: O(V^2). List version uses a min priority queue: O((V+E) log V).
5. Both versions are run 200000 times and the average time is printed.

Program:

```
#include <bits/stdc++.h>
using namespace std;
const int V=6, INF=1e9;
int W[V][V]; vector<pair<int,int>> adj[V];
void dijM(int s,int d[]){          // adjacency matrix  O(V^2)
 bool done[V]={0}; fill(d,d+V,INF); d[s]=0;
 for(int c=0;c<V;c++){
  int u=-1;
  for(int i=0;i<V;i++) if(!done[i]&&(u<0||d[i]<d[u])) u=i;
  done[u]=1;
  for(int v=0;v<V;v++)
   if(W[u][v]&&d[u]+W[u][v]<d[v]) d[v]=d[u]+W[u][v]; } }
void dijL(int s,int d[]){          // adjacency list + min heap
 fill(d,d+V,INF); d[s]=0;
 priority_queue<pair<int,int>,vector<pair<int,int>>,greater<>> pq;
 pq.push({0,s});
 while(!pq.empty()){
  auto [du,u]=pq.top(); pq.pop();
  if(du>d[u]) continue;
  for(auto [v,w]:adj[u])
```

```
    if(du+w<d[v]){ d[v]=du+w; pq.push({d[v],v}); } } }
int main(){
 int e[][3]={{0,1,7},{0,2,9},{0,5,14},{1,2,10},{1,3,15},
          {2,3,11},{2,5,2},{3,4,6},{4,5,9}};
 for(auto&x:e){
  W[x[0]][x[1]]=W[x[1]][x[0]]=x[2];
  adj[x[0]].push_back({x[1],x[2]}); adj[x[1]].push_back({x[0],x[2]}); }
 int d1[V],d2[V]; const int R=200000;
 auto t0=chrono::steady_clock::now();
 for(int i=0;i<R;i++) dijM(0,d1);
 auto t1=chrono::steady_clock::now();
 for(int i=0;i<R;i++) dijL(0,d2);
 auto t2=chrono::steady_clock::now();
 cout<<"Source vertex = 0\nVertex : Distance(Matrix) Distance(List)\n";
 for(int i=0;i<V;i++)
  cout<<" "<<i<<"   :     "<<d1[i]<<"             "<<d2[i]<<"\n";
 double a=chrono::duration<double,nano>(t1-t0).count()/R;
 double b=chrono::duration<double,nano>(t2-t1).count()/R;
 printf("\nAvg time per run (V=6, E=9)\n");
 printf("Adjacency Matrix : %.0f ns\n",a);
 printf("Adjacency List  : %.0f ns\n",b);
}
```

Output:

```
Source vertex = 0
Vertex : Distance(Matrix) Distance(List)
  0    :        0                0
  1    :        7                7
  2    :        9                9
  3    :        20               20
  4    :        20               20
  5    :        11               11

Avg time per run (V=6, E=9)
Adjacency Matrix : 55 ns
Adjacency List   : 128 ns
```

Result: The program was executed successfully and the output is verified.

Program 8

Job Sequencing with Deadlines (Greedy)

Aim:

To implement job sequencing with deadlines using the greedy strategy so that the total profit is maximum.

Algorithm:

1. Sort all jobs in decreasing order of profit.
2. For each job, search for a free time slot from its deadline backwards to slot 1.
3. If a free slot is found, schedule the job there and add its profit; otherwise reject the job.
4. Time complexity is O(n^2) for this simple version.

Program:

```
#include <bits/stdc++.h>
using namespace std;
struct Job{ char id; int dl,pr; };
int main(){
  vector<Job> j={{'a',2,100},{'b',1,19},{'c',2,27},{'d',1,25},{'e',3,15}};
  cout<<"Job  Deadline  Profit\n";
  for(auto&x:j) cout<<" "<<x.id<<"      "<<x.dl<<"        "<<x.pr<<"\n";
  sort(j.begin(),j.end(),[](Job x,Job y){ return x.pr>y.pr; });
  int maxd=0; for(auto&x:j) maxd=max(maxd,x.dl);
  vector<char> slot(maxd+1,'-'); int profit=0;
  for(auto&x:j)
    for(int t=x.dl;t>=1;t--)
      if(slot[t]=='-'){ slot[t]=x.id; profit+=x.pr; break; }
  cout<<"\nSelected job sequence : ";
  for(int t=1;t<=maxd;t++) if(slot[t]!='-') cout<<slot[t]<<" ";
  cout<<"\nMaximum profit = "<<profit<<"\n";
}
```

Output:

```
Job   Deadline   Profit
 a        2         100
 b        1         19
 c        2         27
 d        1         25
 e        3         15

Selected job sequence : c a e
Maximum profit = 142
```

Result: The program was executed successfully and the output is verified.

Program 9

0/1 Knapsack Problem using Dynamic Programming

Aim:

To write a program to solve the 0/1 Knapsack problem using Dynamic Programming.

Algorithm:

1. Let K[i][w] = best profit using the first i items with capacity w. K[0][w] = K[i][0] = 0.
2. If weight[i] <= w: K[i][w] = max(K[i-1][w] , profit[i] + K[i-1][w - weight[i]]), else K[i][w] = K[i-1][w].
3. The answer is K[n][W]. Trace back from K[n][W]: if K[i][w] != K[i-1][w] item i is selected.
4. Time and space complexity: O(nW).

Program:

```
#include <bits/stdc++.h>
using namespace std;
int main(){
 int n=4,W=8, wt[]={2,3,4,5}, pr[]={3,4,5,6};
 int K[5][9]={};
 for(int i=1;i<=n;i++)
  for(int w=0;w<=W;w++){
   K[i][w]=K[i-1][w];
   if(wt[i-1]<=w)
    K[i][w]=max(K[i][w],pr[i-1]+K[i-1][w-wt[i-1]]); }
 cout<<"DP table (rows=items, columns=capacity 0..8)\n";
 for(int i=0;i<=n;i++){
  for(int w=0;w<=W;w++) cout<<setw(3)<<K[i][w];
  cout<<"\n"; }
 cout<<"\nMaximum profit = "<<K[n][W]<<"\nItems selected : ";
 for(int i=n,w=W;i>0;i--)
  if(K[i][w]!=K[i-1][w]){
   cout<<i<<"(w="<<wt[i-1]<<",p="<<pr[i-1]<<") ";
   w-=wt[i-1]; }
 cout<<"\n";
}
```

Output:

```
DP table (rows=items, columns=capacity 0..8)
  0  0  0  0  0  0  0  0  0
  0  0  3  3  3  3  3  3  3
  0  0  3  4  4  7  7  7  7
  0  0  3  4  5  7  8  9  9
  0  0  3  4  5  7  8  9 10

Maximum profit = 10
Items selected : 4(w=5,p=6) 2(w=3,p=4)
```

Result: The program was executed successfully and the output is verified.

Program 10

N-Queens Problem using Backtracking

Aim:

To implement the N-Queens problem using backtracking.

Algorithm:

1. Place queens row by row; col[r] stores the column of the queen in row r.
2. A position (r,c) is safe if no earlier queen is in column c or on the same diagonal (|col[i]-c| = r-i).
3. Try every safe column in the current row and move to the next row. If no column is safe, backtrack to the previous row.
4. When all N rows are filled, a solution is found - print the board.

Program:

```
#include <bits/stdc++.h>
using namespace std;
int N,cnt,col[20];
bool safe(int r,int c){
  for(int i=0;i<r;i++)
    if(col[i]==c||abs(col[i]-c)==r-i) return false;
  return true; }
void solve(int r,bool pr){
  if(r==N){
    cnt++;
    if(pr){
      cout<<"Solution "<<cnt<<":\n";
      for(int i=0;i<N;i++){
        for(int j=0;j<N;j++) cout<<(col[i]==j?"Q ":". ");
        cout<<"\n"; } }
    return; }
  for(int c=0;c<N;c++)
    if(safe(r,c)){ col[r]=c; solve(r+1,pr); } }
int main(){
  N=4; cnt=0; solve(0,true);
  cout<<"Total solutions for N=4 : "<<cnt<<"\n\n";
  cout<<"N  : Number of solutions\n";
```

```
 for(N=1;N<=8;N++){ cnt=0; solve(0,false); cout<<N<<" : "<<cnt<<"\n"; }
}
```

Output:

```
Solution 1:
. Q . .
. . . Q
Q . . .
. . Q .
Solution 2:
. . Q .
Q . . .
. . . Q
. Q . .
Total solutions for N=4 : 2

N  : Number of solutions
1  : 1
2  : 0
3  : 0
4  : 2
5  : 10
6  : 4
7  : 40
8  : 92
```

Result: The program was executed successfully and the output is verified.

Program 11

0/1 Knapsack Problem using Backtracking

Aim:

To use the backtracking strategy to solve the 0/1 Knapsack problem.

Algorithm:

1. Arrange the items in decreasing order of profit / weight.
2. State space tree: at level i choose x[i] = 1 (include) if the weight still fits, or x[i] = 0 (exclude).
3. Bounding function: upper bound = current profit + fractional fill of the remaining items. Kill the node if bound <= best profit so far.
4. When all items are decided, update the best profit and solution vector.

Program:

```
#include <bits/stdc++.h>
using namespace std;
int n=4,M=15,w[]={2,4,6,9},p[]={10,10,12,18};  // sorted by p/w
int best=0,x[4],bx[4];
double bound(int i,int cw,int cp){    // upper bound (fractional)
  double b=cp; int rem=M-cw;
  for(;i<n&&w[i]<=rem;i++){ rem-=w[i]; b+=p[i]; }
  if(i<n) b+=(double)p[i]*rem/w[i];
  return b; }
void bt(int i,int cw,int cp){
  if(i==n){
    if(cp>best){ best=cp; copy(x,x+n,bx); }
    return; }
  if(cw+w[i]<=M){ x[i]=1; bt(i+1,cw+w[i],cp+p[i]); }  // include
  if(bound(i+1,cw,cp)>best){ x[i]=0; bt(i+1,cw,cp); }  // exclude
}
int main(){
  cout<<"Item  Weight  Profit\n";
  for(int i=0;i<n;i++) cout<<" "<<i+1<<"      "<<w[i]<<"       "<<p[i]<<"\n";
  cout<<"Knapsack capacity M = "<<M<<"\n";
  bt(0,0,0);
  cout<<"\nSolution vector x = (";
```

```
 for(int i=0;i<n;i++) cout<<bx[i]<<(i<n-1?", ":")");
 int tw=0; for(int i=0;i<n;i++) tw+=bx[i]*w[i];
 cout<<"\nTotal weight = "<<tw<<"\nMaximum profit = "<<best<<"\n";
}
```

Output:

```
Item  Weight  Profit
 1       2       10
 2       4       10
 3       6       12
 4       9       18
Knapsack capacity M = 15

Solution vector x = (1, 1, 0, 1)
Total weight = 15
Maximum profit = 38
```

Result: The program was executed successfully and the output is verified.

Program 12

Travelling Salesperson Problem using Branch and Bound

Aim:

To implement the Travelling Salesperson problem using the Branch and Bound approach.

Algorithm:

1. Reduce the cost matrix (subtract the row minimum from every row, then the column minimum from every column). Sum of reductions = lower bound of root.
2. To go from city i to city j: cost(child) = cost(parent) + M[i][j] + reduction of the new matrix, where row i and column j are set to infinity and M[j][1] = infinity.
3. Keep all live nodes in a priority queue (least cost first) and always expand the node with the least cost.
4. When a node has visited all cities, its cost is the minimum tour cost.

Program:

```
#include <bits/stdc++.h>
using namespace std;
const int N=5, INF=1e9;
struct Node{ int m[N][N],cost,city,lvl; vector<int> path; };
bool operator>(const Node&a,const Node&b){ return a.cost>b.cost; }
int reduce(int m[N][N]){          // row and column reduction
  int s=0;
  for(int i=0;i<N;i++){
    int mn=INF; for(int j=0;j<N;j++) mn=min(mn,m[i][j]);
    if(mn!=INF&&mn>0){
      s+=mn; for(int j=0;j<N;j++) if(m[i][j]!=INF) m[i][j]-=mn; } }
  for(int j=0;j<N;j++){
    int mn=INF; for(int i=0;i<N;i++) mn=min(mn,m[i][j]);
    if(mn!=INF&&mn>0){
      s+=mn; for(int i=0;i<N;i++) if(m[i][j]!=INF) m[i][j]-=mn; } }
  return s; }
int main(){
  int c[N][N]={{INF,20,30,10,11},{15,INF,16,4,2},{3,5,INF,2,4},
               {19,6,18,INF,3},{16,4,7,16,INF}};
  priority_queue<Node,vector<Node>,greater<Node>> pq;
```

```
 Node r; memcpy(r.m,c,sizeof c);
 r.cost=reduce(r.m); r.city=0; r.lvl=0; r.path={0};
 cout<<"Lower bound at root = "<<r.cost<<"\n";
 pq.push(r);
 while(!pq.empty()){
  Node t=pq.top(); pq.pop();
  if(t.lvl==N-1){
   cout<<"Optimal tour : ";
   for(int v:t.path) cout<<v+1<<" -> ";
   cout<<"1\nMinimum cost = "<<t.cost<<"\n"; break; }
  for(int j=0;j<N;j++) if(t.m[t.city][j]!=INF){
   Node ch=t; ch.path.push_back(j); ch.lvl=t.lvl+1; ch.city=j;
   for(int k=0;k<N;k++){ ch.m[t.city][k]=INF; ch.m[k][j]=INF; }
   ch.m[j][0]=INF;
   ch.cost=t.cost+t.m[t.city][j]+reduce(ch.m);
   pq.push(ch); } }
}
```

Output:

```
Lower bound at root = 25
Optimal tour : 1 -> 4 -> 2 -> 5 -> 3 -> 1
Minimum cost = 28
```

Result: The program was executed successfully and the output is verified.